

バイブコーディング実践講座

Claude Codeの“任せ方”と“検証”の技術

```
claude > 議事録をスライドにまとめて_
```

本日の進め方

全員向け

- AIに“任せて検証する”基本の型
- MCPでNotion・Officeと繋ぐ日常活用
- 情報を守る3層のルール
- 明日からの始め方

エンジニア向け

- CLAUDE.md・skills・コマンドの設計
- “頭が悪くなる”仕組みと対処法
- 権限モードと開発フローの型
- 実体験の失敗談と効率化の習慣

今日のゴール

- 1 使ってみたくなる
- 2 任せ方と守り方がわかる
- 3 自分の業務での使い道が見える

バイブコーディングの“誤解”と“本質”

△ 惜しい使い方

“雰囲気丸投げ”

実は、丸投げでもある程度のものは出来てしまう。ただ品質と再現性は運まかせで、複雑になるほど崩れる。

○ 本質

AIを理解し、任せて、検証する

仕組みを理解して任せ、出力を検証する。同じAIでも、成果の質とスピードはここで差がつく。

今日の裏テーマ 丸投げで“動く”から、理解して“使いこなす”へ。

CLAUDE.md vs コマンド vs skills

CLAUDE.md

常に読み込まれる前提・ルール。
技術スタックや規約を短く書き、
毎回の会話の土台になる。

```
CLAUDE.md
```

コマンド

自分で呼び出すショートカット。
短い定型作業を一発で実行する。

```
/log ・ /push
```

skills

必要な時だけ読み込まれる作業の
型。複雑な手順や判断をまとめて
再利用。コマンドとして呼び出す
こともできる。

```
implement / review
```

使い分け : いつも守る前提 → CLAUDE.md / 短い定型 → コマンド / 判断や長い手順 → skills。

これから順に、3つの道具の中身と使い分けを見ていきます。

CLAUDE.md / コンテキスト設計 — すべての土台

./CLAUDE.md — プロジェクト直下に置く

```
# 概要 / スタック
- Rails 8 / Next.js / MySQL
# 規約・コマンド
- 命名: camelCase
- test: bundle exec rspec
# 詳細は docs/ に逃がす
→ docs/backend.md
→ docs/frontend.md
```

「前提」だけを短く渡す

書くこと：

概要・技術スタック・規約・コマンド・禁止事項。常に効く「前提」だけ。

詳細は docs/ へ逃がす：

機能ごとの仕様は docs/*.md にリンク。使う時だけ読み込む（progressive disclosure）。

長さの目安：

数十行に抑える。長いほど毎回コンテキストを消費する（コンテキスト税）。

読み込まれるタイミング

Claude Code がその会話を立ち上げた時点で自動読込。「/clear」や「/resume」など会話の区切りで読み直される（会話の途中で書き換えても自動反映されない）。

おすすめスラッシュコマンド

セッション中に「/」で呼び出す。知っているだけで効率と精度が変わる 6 つ

/effort

推論量を調整（low～ultracode）。要件定義・仕様書を固める作業は高めが効く

/btw

履歴を汚さずに横道の質問。文脈を保ったまま、ちょっと聞きたい時に

/resume

過去の会話を開き直して再開。続きから作業を再開できる

/clear

文脈を空にして新規開始。動作が軽くなる（プロジェクトメモリは保持）

/rewind

会話・コードをチェックポイントまで巻き戻す。指示ミスのリカバリーに

/insights

過去セッションを分析し、使い方の傾向と改善点をレポート

TIP 迷ったら / で全コマンド一覧。長くなったら /context で消費を確認 → /compact で要約圧縮

実際の画面：/effortで推論量の切り替え

MEMORISE#2

packages

backend

log

spec

factories

models

requests/graphql

admin_login_spec.rb

admin_me_spec.rb

health_spec.rb

login_spec.rb

logout_spec.rb

me_spec.rb

public_wordbook_spec.rb U

public_wordbooks_spec.rb U

sign_up_spec.rb

support

rails_helper.rb

spec_helper.rb

tmp

.dockerignore

アウトライン

タイムライン

VS CODE POKÉMON

DEPENDENCIES

packages > backend > spec > requests > graphql > public_wordbook_spec

```
1 # frozen_string_literal: true
2
3 require "rails_helper"
4
5 RSpec.describe Resolvers::PublicWordbook do
6   let(:user) { create(:user) }
7   let(:query) do
8     <<~GQL
9     query($id: ID!) {
10       publicWordbook(id: $id) {
11         id title label part
12         children { id title part wordsCount
13           words { id question answer } }
14       }
15     }
16   end
17
18   describe "正常系" do
19     it "公式の親と、その子（章）・単語を読み取り専用で返す" do
20       parent = create(:wordbook, :official,
21         title: "TOEIC", label: "toEIC")
22       chapter = create(:wordbook, :official,
23         title: "Day1", part: "1", parent_id:
24         parent.id, order_index: 1)
25       create(:word, wordbook: chapter, question:
26         "apple", answer: "りんご")
27     end
28   end
29 end
```

Commit lib/tasks additions only

一旦semi-authは使用しない設計で進めてほしいです。

```
IN  cd "c:/Users/sound/Data/my-project/MemoRise#2" && APP="packa
OUT  --- result ---
packages/frontend/src/app/(auth)/basicWord/[parentId]/page.tsx
packages/frontend/src/app/(auth)/basicWordList/page.tsx
```

Now I'll up Modes ⬆ + tab to switch

Read c:¥Us rc¥app¥(aut) Ask before edits
Claude will ask for approval before making each edit

Read c:¥Us rc¥app¥(aut) Edit automatically
Claude will edit your selected text or the whole file

Edit c:¥Us ¥app¥(auth) Plan mode
Claude will explore the code and present a plan before editing

Edit c:¥Us Auto mode
Claude will automatically choose the best permission mode for each task

ctrl esc to focus Effort (Extra high)

+ public_wordbook_spec.rb

Auto mode

AI に自分の使い方を分析させてみた

実際の /insights レポート (2026-05-15 ~ 06-26 / 86 セッション・1,524 メッセージを分析)

1,524

メッセージ

86

セッション

388

ファイル

+15,815 / -705

行 (add/del)

58.6

msgs / 日

○ 効果のある点

出荷と学習を両立。実装しながら Rails/GraphQL の概念も確認し、BE ~ E2E ~ PR まで縦に貫通。/log・/report・/dev を自作し、Claude を自分の流儀へ形作っている

△ 足かせ・課題

Claude 側：時々スコープ超過（不要ファイル作成・聞いただけでリファクタ）。自分側：PowerShell 構文が Bash コミットに混入。「説明だけ / 実装」の境界が曖昧

▶ 試せるクイックウィン

実証済みフロー（Rails+GraphQL+React+RSpec+E2E+PR）を /feature スキル化。PreToolUse フックでコミット整形。GitHub MCP で Issue / PR を円滑に

▶ 野心的なワークフロー

/dev を Issue→PR の自律パイプラインへ。テストがグリーンになるまでループ。リファクタは worktree で並列サブエージェントを競わせ、最良案を自動採用

POINT 使えば使うほど、強みと改善点・次の一手まで AI が言語化してくれる

個人skillsの紹介 — 自作の例

skill = “決まった作業手順” をAIに覚えさせる仕組み

/implement

実装専任。要件に沿って実装し、実装後に自己レビュー。

/review

差分レビュー専任。リードオンリー（書き込み権限なし）。

/report

出力フォーマットの本体。サマリ+変更ファイル表で整形。

共通ルール：説明を具体的に（他skillと判別しやすいようにする） / 権限は最小に / 出力を固定する

読み込まれるタイミング 起動時は名前（name）と説明（description）だけ（CLAUDE.mdと同じタイミング）。
使うと判断された時に本文が読み込まれる。

実物を見る — review skill の中身

review / SKILL.md

```
---
name: review
description: このブランチの差分を
  レビューするスキル(リードオンリー)。
  ...「差分を確認して」で使う。
allowed-tools: Read, Grep, Glob,
  Bash(git diff:*), Bash(git status:*),
  Bash(git merge-base:*), ...
---
# 差分レビュー (リードオンリー)
コードは書き換えない。実装が必要なら
implement スキルを使う。
```

description に注目

この説明文で“いつ起動するか”が決まる。起動時に読まれるのはここだけ（プログレッシブ・ディスクロージャー）。

allowed-tools に注目

Edit / Write が無い = リードオンリー。レビューが誤ってコードを書き換える事故が“原理的に”起きない（最小権限）。

※ 手順・レビュー観点などの本文は省略。GitHub かその場でお見せできます。

ファイル(F)編集(E)選択(S)表示(V)移動(G)実行(R)ターミナル(T)ヘルプ(H)

プレゼンテーション

SKILL.md 3 X

skills > report > SKILL.md > ...

SKILLS

implement

SKILL.md

report

SKILL.md 3

review

claude-code-presentation.p...

1 ---

2 name: implement

3 description: 要件定義issue(マークダウン)が渡されたら、それに沿って実装するスキル(実装専任)。引数がissue番号/パスならそれを読み込み、本文が貼られていればそれを使う。そのプロジェクトの既存規約に合わせて実装し、実装後は開発内容のテストを新規作成し、新規テストと変更箇所に関連するテストのみ実行(全体テストはCI等に委ねる)、既存ドキュメント(API仕様・ディレクトリ構成等のmd)があれば実装に追従させて最新化し、自己レビューしてから report の形式で報告する。差分のレビューだけしたい場合は review スキルを使う(このスキルは実装しないと起動しない)。

4 allowed-tools: Read, Edit, Write, Grep, Glob, Skill, Bash(git diff:*), Bash(git status:*), Bash(npm test:*), Bash(npm run test:*), Bash(yarn test:*), Bash(pnpm test:*), Bash(npx jest:*), Bash(npx vitest:*), Bash(npx mocha:*), Bash(npx tsc:*), Bash(npx playwright test:*), Bash(bundle exec rspec:*), Bash(rspec:*), Bash(pytest:*), Bash(python -m pytest:*), Bash(go test:*), Bash(cargo test:*)

5 ---

6 # 実装(専任)

7

8 要件定義issue(マークダウン)が渡されたときに、それを実装する。****実装しない依頼(差分レビューのみ)では起動しない**** → それは `review` スキルの担当。

9

10 ## 手順

11 1. 要件を読む。引数がissue番号/パスなら Read、本文が貼られていればそれを使う。

12 2. 対応タスクを箇条書きで整理して提示する。

13 3. 要件に沿って実装する。そのプロジェクトの既存の規約・周辺コード・使用言語/フレームワークに合わせる。****要件に書かれていない変更は入れない(テストの新規作成は除く。手順4を参照)。****

14 4. ****開発した内容を検証するテストを新規作成する。*** まず `package.json` の scripts / `Gemfile` / 設定ファイル(`jest.config`・`vitest.config`・`pytest.ini` 等) / CI設定を見て、そのプロジェクトのテストランナー・実行方法・配置規約を判定する(推測でコマンドを打たない)。その上で、実装した挙動・修正した不具合をカバーするテストを、そのプロジェクトのテスト規約(配置・命名・フレームワーク)に合わせて書く。****テストフレームワークが無いプロジェクトでは無理に導入しない****(スコープ膨張のため)。その場合はテストを作らず、手順8で report のテスト項目に「テスト基盤なし」と理由付きで記す。

15 5. ****テストを実行する(関連範囲に限定)。**** 関連テストは、変更ファイルと命名/配置規約で対応するもの(例: `foo.ts` → `foo.test.ts`、`app/models/x.rb` → `spec/models/x\_spec.rb`)や、変更対象を import/参照しているテストから特定する。手順4で作った新規テストと、それら関連する既存テストのみを走らせ、通ることを確認する。落ちたら実装またはテストを直して、通るまで再実行する。****プロジェクト全体のテストは毎回実行しない****(開発スピード優先。全体の回帰確認はCIや別途の `verify` に委ねる)。実行は手順4で判定したランナーを使い、ファイル/パス指定で範囲を絞る(例: `npx jest path/to/file.test.ts` / `bundle exec rspec spec/xxx\_spec.rb` / `pytest tests/test\_xxx.py` / `npx tsc --noEmit` は変更範囲の型確認として可)。****判定したランナーが allowed-tools の許可コマンドに無いプロジェクトでは****、テスト実行は承認プロンプト経由になるが対象外とし、無理に実行しない。その場合は手順8で report のテスト項目にその旨(未実行の理由)を記す。

16 6. ****既存ドキュメントの同期。*** リポジトリに API 仕様・ディレクトリ構成・設計などを記した md ドキュメント(例: `README`・`docs` / `API.md`・OpenAPI/スキーマ説明)があり、今回の実装がその記載内容を変える場合は、****該当箇所を新規コードに合わせて最新化する****(追加/変更したエンドポイント・型・引数・ディレクトリ・設定など)。該当ドキュメントが無ければ何もしない(****新規には作らない****)。

17 7. 自分の差分を `git diff` / `git status` で読み直し、要件と1:1で対応しているか ****自己レビュー**** する。新規テストが****実装を実際に検証しているか****(トリビアルな assert・常に通るだけのテストになっていないか)、および ****ドキュメントの記載が実装と食い違っていないか**** もここで確認する。

18 8. ****報告は `report` スキルを呼び出して行う****(Skill ツールで report を起動し、整形は report に任せ

1 ---

2 name: report

3 description: 変更内容を決まった形式で報告するためのフォーマット係スキル。実装やレビューの結果を「サマリ → 変更ファイルの表(バックエンド/フロントエンドに分割) → テスト(実行した場合のみ)」の形式で出力する。バックエンド=サーバーサイド、フロントエンド=クライアントサイドとして分類する。変更を報告したい・レポートしたいときに使う(implement/review 経由でも、直接編集した変更を単独で報告する用途でも使える。出力フォーマットの唯一の持ち主)。

4 allowed-tools: Read, Grep, Glob, Bash(git diff:*), Bash(git status:*)

5 ---

6 # 変更レポート(表形式) - 出力フォーマットの本体

7

8 このスキルは ****出力フォーマットの唯一の持ち主****。実装(implement)やレビュー(review)から呼ばれてその結果を整形するほか、****それらを経由しない直接の編集(手作業の変更・他の作業)でも単独で起動でき****、変更を報告したいときはいつでもこの形式で出力する。

9 このスキル自身はコードを書き換ええない(報告専任 = リードオンリー)。****単独起動時****は、変更ファイルを `git diff` / `git status` から自分で洗い出して表を作る(テスト項目は、テストのコマンド/結果が渡された場合のみ埋め、無ければ省略する)。

10

11 ## 出力フォーマット(サマリと変更ファイルは必ず出す。テストは実行した場合のみ)

12

13 ### 1. サマリ

14 要件を実装した場合は何をしたか、差分レビューの場合は何が変わっているかを2~4文で。

15

16 ### 2. 変更ファイル(バックエンド / フロントエンドで分けて表化)

17 変更したファイルを ****バックエンドとフロントエンドに分け****、それぞれ2カラムの表で出力する。

18

19 - 分類の目安: バックエンド = サーバーサイド(API・DBアクセス・ビジネスロジック等。例: `.rb` / `.go` / `.py`、`config`、GraphQL スキーマ)。フロントエンド = クライアントサイド(UI・コンポーネント・スタイル等。例: `.tsx` / `.ts` / `jsx` / `vue`、CSS)。

20 - どちらにも当てはまらないもの(設定・インフラ・スクリプト等)は「その他」の表にまとめる(無ければ省略)。

21 - プロジェクトの構成に合わせて分類軸は調整してよい。

22 - 各表とも「ファイル」列 = 相対パスへの markdown リンク `[ファイル名](相対パス)`、「内容」列 = 変更内容を1行で。

23 - 片方に変更が無い場合、その表は省略してよい。

24

25 #### バックエンド

26 | ファイル | 内容 |

27 |---|---|

28 | [ファイル名](相対パス) | 変更内容を1行で |

29

30 #### フロントエンド

31 | ファイル | 内容 |

32 |---|---|

33 | [ファイル名](相対パス) | 変更内容を1行で |

34

35 ### 3. テスト(テストを実行した場合のみ)

36 実装(implement)でテストを新規作成・実行した場合に、この「テスト」項目でまとめる。テストを実行していない場合(差分レビューのみ等)は省略する。ただし次の未実行ケースは、テスト項目に理由付きで1行記す: テスト基盤が無いプロジェクトの場合は「テスト基盤なし」と理由付きで記す。

0 3 6

行 5、列 4 スペース: 2 UTF-8 LF {} Skill 8 Go Live Prettier

個人skills vs プロジェクトskills

個人 skills

自分の作業フロー。どのプロジェクトでも持ち歩く。

例) implement / review / report

置き場所 ~/.claude/skills/ (ホーム配下)

プロジェクト skills

チーム共有の規約・レビュー観点。リポジトリにコミットして全員で使う。

置き場所 .claude/skills/

⚠ 依存の向きに注意 : プロジェクト → 個人 への依存はNG (チームがその個人skillを持っていない=壊れる)。

世の中の定番skill — 公式 anthropics/skills

自作だけじゃない。公式リポジトリ anthropics/skills（13万★超）に実用skillが多数。中でもエンジニアにおすすめの2つ。

skill-creator

skillを作る・磨くためのskill

新規作成から、説明文(description)の最適化、トリガー精度の検証まで支援。“作る側”を底上げしてくれる。

webapp-testing

AIがブラウザを操作して動作検証

Webアプリを自動操作してテスト。今日のテーマ“検証”の最上位。フロントの動作確認を丸ごと任せられる。

他にも **pptx / docx / xlsx / pdf**（資料）、**mcp-builder**（MCP生成）など全17種。

導入： `/plugin marketplace add anthropics/skills` → `install` して名前と呼ぶだけ

失敗談 — skillの重複で遅くなった

起きたこと 個人skillを増やすうち、似た説明・重複した手順を複数のskillに持たせてしまった。

例) 要件に合わせてコーディングするskillとレビューするskillが競合して、レビュー（read-only）途中でリファクタリングしたりするようになった。

原因

skillは毎ターン description で「使うべきか」を判断する。重複・類似があるとAIが迷い、判断が増えて出力が遅くなった。

学び

1 skill = 1役割。説明は具体的かつ排他的に。重複は統合し、数も絞る（context tax）。権限ごとにskillsを分けることも重要。

skillは“足す”より“重複を消す”。少数精鋭のほうが速い。

私の開発フロー全体像



鍵： /implement ・ /report ・ /review は skill で半自動化。コード確認とテスト実行が“人間のチェックポイント”（強調部）。丸投げで終わらせない。

実際の画面 : VS Code × Claude Code

MEMORISE#2

packages

backend

db

log

spec

factories

models

study_detail_spec.rb

study_record_spec.rb

user_spec.rb

user_word_tag_spec.rb

user_wordbook_progress_spe...

word_spec.rb

wordbook_spec.rb

requests

support

rails_helper.rb

spec_helper.rb

tmp

アウトライン

タイムライン

VS CODE POKÉMON

DEPENDENCIES

```
packages > backend > spec > models > study_detail_spec.rb
1 require "rails_helper"
2
3 RSpec.describe StudyDetail, type: :model do
4   it { is_expected.to belong_to(:study_record) }
5   it { is_expected.to belong_to
      (:chapter_wordbook).class_name("Wordbook").
      optional }
6
7   describe "バリデーション" do
8     it { is_expected.to validate_presence_of
          (:total_count) }
9     it { is_expected.to validate_presence_of
          (:correct_count) }
10  end
11
12  describe "correct_not_exceed_count" do
13    it "正答数が問題数を超えると無効" do
14      detail = build(:study_detail, total_count:
15        5, correct_count: 6)
16
17      expect(detail).to be_invalid
18      expect(detail.errors[:correct_count]).to
19        be_present
20
21    end
22
23    it "正答数 == 問題数は有効" do
24      detail = build(:study_detail, total_count:
25        5, correct_count: 5)
26      expect(detail).to be_valid
27    end
28  end
29 end
```

Rebuild v1 schema with R... query_type.rb 20: 🔒 ...

Rebuild v1 schema with Rails models

</> c:\Users\sound\Data\my-pr...

ok. dbフォルダを確認してください。問題がなさそうだったらコミットします

レビュー結果 (db フォルダ)

対象	判定	確認内容
schema.rb	✓	wordbooks に deleted at / kind (default personal) が反映。カラムはアルファベット順で正しく、index wordbooks on deleted at も追加済み。FK (users / 自己参照 parent) も整合
seeds.rb	✓	kind = :official に修正済み・冪等 (find or create by!)。admin は role 判定で既定単語帳が作られず、公式単語は user_id なしで user consistency を満たす
migrate/ (7本)	✓	各テーブル定義・制約・FK の on delete (cascade / nullify) とも適切

schema.rb は migration から再生成しても同じになる状態 (カラム順・index・FK 一致) なので、db:migrate で差分が出る心配もあ

ctrl esc to focus or unfocus Claude

+ 📄 | study_detail_spec.rb ⚡ Auto mode ⬆

コマンド vs ファイル – 2つの実行スタイル

同じ Claude Code でも「ターミナル(CLI)」と「IDE 拡張パネル（ファイルを開いて作業）」で得意分野が違う

ターミナル (CLI)

下のターミナルで対話実行

○ メリット

- 環境非依存：VS Code 以外でも同じ操作で動く
- 起動が速く軽量・キーボードで完結
- コマンド / 履歴 / CI 連携・自動化に強い

△ デメリット

- 差分や全体像が視覚的に見えにくい
- 複数ファイルのレビューは窮屈

IDE パネル / ファイル

右側パネルでファイルを見ながら

○ メリット

- 開いているファイル / 選択範囲が自動で文脈に
- 差分(diff)を画面上で確認・承認できる
- エディタと並べて直感的にレビュー・編集

△ デメリット

- VS Code 環境に依存する
- パネルが画面を占有し・やや重い

使い分け 繰り返し・自動化・CI は **コマンド(CLI)**、差分を見ながらのレビュー・編集は **IDE パネル** が向く

「Claude Codeが頭が悪くなる」の正体

原因：コンテキストの汚染

会話が長くなるほど、古い情報・無関係な話が文脈に混ざる。AIは会話を全部読み続けるので、雑多になるほど精度が落ちる。

**比喻：散らかった机では
良い仕事はできない**

症状

指示を無視しはじめる

前の間違いを繰り返す

的外れな出力が増える

→ 賢さが落ちた？ まず“机が散らかっている”を疑う。

対策：/clear で会話を区切りながら進める —— 話題が変わったら惜しまずリセット。長い1スレッドで粘らない。

開発の権限について

AIに何を許可するかは、**自分で設計できる**（allowed-tools / 許可制）。画面のモード（Ask / Auto など）はその場の切り替え、`settings.json` は毎回効く恒久ルール、という関係です。

原則：最小権限

review skill は **読むだけ**。Edit / Write 権限を持たせない。権限でskillを分けるとよい。

→ レビューのつもりが勝手にファイルを書き換える事故を、権限レベルで防ぐ。（先ほどの review の実物がこれ）

トレードオフ

自動実行は便利。でも全部を自動許可にすると危ない。「何を許すか」を意識的に決めるのが大事。

便利さとリスクはトレードオフ。権限は“意識して絞る”。

権限モードの使い分け

編集をどこまで自動で許すか。作業のリスクとスピードに応じて切り替える（Shift + Tab）

Modes

Shift + Tab to switch

Ask before edits

Ask for approval before each edit



Edit automatically

Edit selected text or the whole file

Plan mode

Explore the code and present a plan first

Auto mode

Auto-pick the best permission per task

Effort Extra high ●●●●○

4 つのモードと、それぞれの役割

Ask before edits（既定）

1 件ずつ承認してから編集。最も安全

Edit automatically

選択範囲やファイルを、確認なしで編集

Plan mode

まず計画を提示。着手前にすり合わせ

Auto mode

タスクごとに最適な権限を自動で選択

私の使い分け

/implement で一括開発を始める時は **Auto mode**。自分でコードレビュー・リファクタする時は **Ask before edits** で 1 件ずつ確認。

セキュリティ / 情報の扱い

渡してはいけない情報

- 顧客情報・個人情報
氏名・メール・取引内容など。マスクしてから渡す
- 社内固有名詞・社内コード
社名・プロダクト名は一般名詞に置き換える
- APIキー・認証情報
.env・環境変数で管理。チャットに直接貼らない
- 内部URL
社内システムの構成が推測できる情報も同様に扱う

仕組みで守る・保つ

設定でブロックし、こまめにリセットする。

① settings.json で .env を deny、 CLAUDE.mdで制御

```
permissions.deny: ["Read(.env)"]
```

② /clear で会話を定期リセット

長い会話ほど精度が落ちる → 新規化で“頭が悪くなる”を防ぐ

③ CLAUDE.md に守りたいルールを明記

禁止事項・機密の扱い方を常時ルール化して念押し

会社のネットワーク／データ方針に従う。便利さの裏で「何を外に出すか」を常に意識。

鉄則：迷ったら出さない —— 判断に迷う情報は、まず出さない。それだけで事故の大半は防げる。

MCP × 日常ツールで、雑務が消える

いつものツール（Gmail・カレンダー・Slack）に "AIの手" が届く。コードは書かない、話しかけるだけ。

イチオシ

朝の秘書ダッシュボード

Gmail + カレンダー + Slack

出社時に "今日の予定・要返信メール・自分宛メンション" を Slack へ1通で自動お届け。
あちこち開かなくても、朝の状況把握が 2〜3 分で終わる。

「まとめて送って」

受信トレイから重要メールだけ整理し、返信の下書きまで用意

メール → カレンダー

メール内の日程を抽出して登録（登録前に一覧で確認）

Slack × カレンダー

メンションと予定から、案件別・優先度付きの TODO を生成

議事録・日報の自動化

予定とメモから下書きを作って Slack へ。書く手間がほぼ消える

POINT

読み取り（探す・まとめる）は任せて自動、書き込み（登録・送信）は "確認してから"。コードを書かない人こそ、恩恵が一番大きい。

MCP連携の実例 — Notion・Office活用

MCP = 外部サービスとの“接続口”。自分の業務ツールに、AIの手が届く。

技術ナレッジ庫

学びをClaudeがNotionに記録・自動分類。あとで検索・閲覧できる“第二の脳”（cc-tech-library）。

就活の選考管理

企業・志望度・メモをNotionで一元管理。Claudeに相談しながら更新できる。

Office資料の作成

WordやPowerPointをMCPで接続。AIと対話しながら文書やスライドを簡単に作成できる。

例：「今日の学びをライブラリに記録して」→ Claudeが分類してNotionに追加。

例：「この資料の修正案をスライドに反映して」→ ClaudeがPowerPointを直接編集。

自分の“第二の脳”をClaudeと育てる。記録も整理も“ついで”で終わる。

明日から始める手順

1 触ってみる

小さなタスクで1つ試す。完璧な設定より“1回試す”。

2 文脈を渡す

CLAUDE.md か簡単なメモでプロジェクトの前提を渡す。

3 困ったら

/clear・計画→実行・小さく刻む を思い出す。

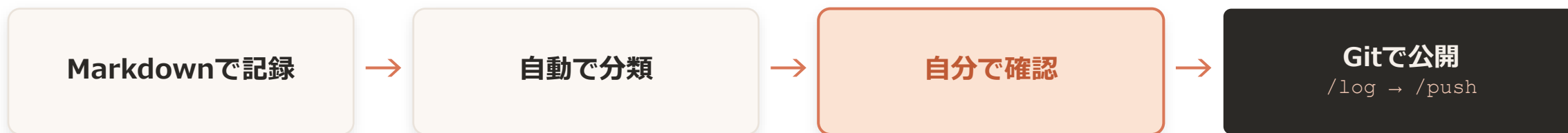
4 慣れたら

自分のskill / コマンドを1個作ると世界が変わる。

ハードルは低く。まずは小さく始めるのが一番のコツ。

ポートフォリオ : cc-tech-library

インターン・学習の技術知見を、Claude Codeで最小コスト運用する個人ナレッジ庫



詰まっている要素

今日話した skill / コマンド / セキュリティが、全部入った実物。

機密の扱い

公開リポゆえ、社内情報は書かず「技術概念」レベルに抽象化して記録。

github.com/mikann-code/cc-tech-library

社内で広げる提案 / 参考リンク

チームで効かせる

- 共通の CLAUDE.md / プロジェクトskillを整える
- チームのレビュー観点をskill化して底上げ
- 公式 anthropics/skills を社内マーケットプレイスに
- 成功事例・失敗事例を共有する場をつくる

参考

- 公式ドキュメント (Claude Code)
code.claude.com/docs/ja
- anthropics/skills (公式skill集)
github.com/anthropics/skills
- cc-tech-library (本日の実例)
github.com/mikann-code/cc-tech-library

個人で終わらせず、組織の資産にする。

まとめ — 持ち帰り3つ

1

道具を整える — CLAUDE.md ・ skills ・ コマンド

前提はCLAUDE.mdに、手順はskillsに、定型はコマンドに。“任せ方”は仕組みにできる。

2

AIの仕組みを知る — 賢さはコンテキストで決まる

“頭が悪くなる”の正体は文脈の汚染。こまめにリセットし、出力は必ず人間が検証する。

3

安全に広げる — 権限 ・ セキュリティ ・ MCP

権限モードで任せ方を調整し、迷ったら出さない。MCPで開発から日常業務まで、非エンジニアも“作れる”。